

SPSC 无锁环形缓冲区

内存序、缓存索引优化与无符号溢出正确性的第一性原理论述

Robert (Bo) Hou · Robert's Technical Notes · 2026 年 6 月

摘要。单生产者单消费者(SPSC)无锁环形缓冲区是低延迟系统的核心数据骨架。本报告从第一性原理出发,分三个层次展开设计。第一,推导每个原子操作所需的最小内存序,并建立一个精确判据:何时必须用 acquire 加载、何时 relaxed 即足够——这一区分在实践中常被过度保守地指定。第二,分析缓存索引优化,通过单调性论证其安全性,并澄清它降低的是一致性开销,而非正确性或填充速率。第三,给出一个基于同余理论的形式化证明,说明绝对计数设计在无符号硬件溢出下仍然正确,并以不变量维持归纳法加以加固。

1. 基础设计与记号

我们将可平凡复制的 T 类型元素存入大小为 Capacity(2 的幂)的定长数组中。两个单调递增的绝对计数器——tail_(仅生产者写)与 head_(仅消费者写)——通过位掩码索引缓冲区。

```
template <typename T, size_t Capacity>
class SpscQueue {
    static_assert((Capacity & (Capacity - 1)) == 0,
                  "Capacity must be power of 2");

    alignas(64) std::atomic<size_t> head_{0}; // consumer writes
    alignas(64) std::atomic<size_t> tail_{0}; // producer writes
    alignas(64) std::array<T, Capacity> buf_;

public:
    bool try_push(const T& item) { // producer only
        size_t t = tail_.load(std::memory_order_relaxed); // [A]
        size_t h = head_.load(std::memory_order_relaxed); // [B]
        if (t - h == Capacity) return false; // full
        buf_[t & (Capacity - 1)] = item;
        tail_.store(t + 1, std::memory_order_release); // [C]
        return true;
    }

    bool try_pop(T& out) { // consumer only
        size_t h = head_.load(std::memory_order_relaxed); // [D]
        size_t t = tail_.load(std::memory_order_acquire); // [E]
        if (h == t) return false; // empty
        out = buf_[h & (Capacity - 1)];
        head_.store(h + 1, std::memory_order_release); // [F]
        return true;
    }
};
```

三个结构性决策需要在开篇强调。

- 绝对计数,而非回绕索引。head_ 与 tail_ 永远只增不减;物理槽位是 counter & (Capacity-1)。这使占用量 $t - h$ 成为一次减法,无需任何分支处理回绕。
- 2 的幂容量。掩码 $\& (Capacity-1)$ 恰好等于 $\text{mod } Capacity$,消除了一次除法。
- 按缓存行对齐。head_ 与 tail_ 占据不同的缓存行($\text{alignas}(64)$)以避免伪共享:生产者只写 tail_,消费者只写 head_;若把二者放在一起,即便它们逻辑上互相独立,该缓存行也会在两核间因 MESI 协议而反复弹跳(ping-pong)。

2. 内存序:最小规格

2.1 两个独立的事实:原子性 vs. 顺序性

必须区分 `std::atomic` 提供的两类保证。

- 原子性:原子加载/存储不可分割(不会读到撕裂值),且对任何单个原子变量,所有线程都观察到一致的修改顺序(modification order)——即便在 relaxed 下也是如此。
- 顺序性:针对不同变量的操作,是否以某种特定顺序对另一线程可见。这才是 memory_order 所控制的东西。

relaxed 操作给你前者,但不给后者。这正是为何读取自己的计数器用 relaxed 永远安全:凭借“读到自己最新写入(read-your-own-write)”加上单变量修改顺序的一致性,生产者对 tail_ 的 relaxed 加载 [A] 与消费者对 head_ 的 relaxed 加载 [D],总能观察到各自的最新存储。

2.2 release/acquire 握手

- release(发布方):在宣告“就绪”的那个存储上,所有序在其前的写入,都保证对任何随后读到该宣告值的线程可见。它是一道单向的“向下”栅栏——前面的写入不能下沉到它之后。
- acquire(读取方):在读到该宣告值的那个加载上,所有序在其后的读取都不能被提到它之前。它是一道单向的“向上”栅栏。

握手仅当 acquire 加载真正读到了 release 存储所写的值时才触发。在那一刻,一条 synchronizes-with 边被建立,发布方在其 release 之前写的一切,都对读取方在其 acquire 之后变得可见。

注记。 release/acquire 不必“对称”。一个 release 存储与一个 relaxed 加载配对仍然合法;那个 relaxed 加载只是不享受同步而已。一个加载是否需要 acquire,取决于该线程在加载之后接下来做什么,而非取决于对面的存储是不是 release。

2.3 决定性判据

命题(acquire 判据)。 对一个原子标志的加载需要 acquire 序,当且仅当:该线程在读到标志之后,接着去读取另一块由发布方在其对应 release 存储之前写入的内存区域。若加载到的值仅用于算术或控制决策,且之后不再读取任何由生产方写入的载荷,则 relaxed 即足够。

将此判据应用于两个跨线程加载。

[E] — 消费者加载 `tail_`: 必须 `acquire`。加载之后, 消费者执行 `out = buf_[h]`, 读取的是生产者在其 `release` 存储 [C] 之前通过 `buf_[t] = item` 填入的槽位。关键在于: 地址 `buf_[h]` 依赖的是 `h` (消费者自己的计数器), 而非加载得到的 `t`。因此并没有数据依赖强制“读载荷”等待“加载 `tail_`”; 二者之间只隔着一条经由 `if (h == t)` 的控制依赖, 而控制依赖并不能阻止投机执行。CPU 可能在加载尚未完成前就投机地执行 `out = buf_[h]`, 读到陈旧或写了一半的数据。[E] 上的 `acquire` 禁止这种重排; 配合 [C] 的 `release`, 它保证一旦观察到新的 `t`, `buf_[h]` 必已被完整写入。

[B] — 生产者加载 `head_`: `relaxed` 即足够。加载之后, 生产者计算 `t - h` 以判断是否已满, 然后写入 `buf_[t]`。它从不读取任何由消费者写入的载荷 (消费者只是消费; 它不会往 `buf_` 里写任何生产者要读的东西)。因此无需 `acquire`: `head_` 的值纯粹被当作一个算术量使用。

注记 (为何陈旧的 `head` 不会破坏正确性)。设 [B] 为 `relaxed`, 生产者读到一个过时的 `head_`。因为 `head_` 单调递增, 过时值严格 $\leq$ 真实值, 于是生产者低估了可用空间。最坏的结果是: 在其实有空位时给出一次虚假的“已满”; 生产者绝不会在没有空位时误以为有空位。误差永远偏向保守一侧, 故正确性得以保持。第 3 节的缓存索引优化依据的正是同一论证。

2.4 六个内存序汇总

位置	操作	内存序	理由
[A]	<code>tail_.load(())</code>	<code>relaxed</code>	读到自己最新写入
[B]	<code>head_.load(())</code>	<code>relaxed</code>	值仅用于算术
[C]	<code>tail_.store</code>	<code>release</code>	发布 <code>buf_</code> 载荷
[D]	<code>head_.load(())</code>	<code>relaxed</code>	读到自己最新写入
[E]	<code>tail_.load(())</code>	<code>acquire</code>	之后要读载荷 <code>buf_[h]</code>
[F]	<code>head_.store</code>	<code>release</code>	读完成后才释放槽位

唯一承重的握手是 [C] `release` $\leftrightarrow$ [E] `acquire`。[F] 之所以是 `release`, 不是为了与 [B] 配对, 而是为了保证消费者的读取 `out = buf_[h]` 在它把槽位宣告为可复用之前完成; 否则生产者可能覆盖一个仍在被读取的槽位。

2.5 平台落地

在 x86(TSO) 上, `relaxed/acquire/release` 这一整族加载与存储都编译为普通的 `mov`; 硬件本就禁止除 `store` $\rightarrow$ `load` 之外的所有重排。只有 `seq_cst` 存储会带来额外开销 (一条 `xchg` 或 `mov+mfence` 以排空 `store buffer`), 这正是低延迟代码避开 `seq_cst` 的原因。在弱序架构 (ARM、POWER) 上, `acquire/release` 会发出真正的屏障指令, 因此在那里把 [B] 设为 `relaxed` 能省下一条真实的栅栏。该抽象是可移植的: 工程师只需就“数据 + 旗子”的 `happens-before` 进行推理, 编译器会把它下降为各平台正确的指令。

3. 缓存索引优化

3.1 动机:一致性是正确的,但不是免费的

每一次加载 `head_` 的 `try_push` 都要付出跨核成本。消费者最近写过 `head_`,故其缓存行在消费者核上处于 `Modified` 状态。生产者读它时,MESI 必须跨核转移所有权——几十到上百个周期——而随后消费者的写又会夺回该行,造成 ping-pong。MESI 保证生产者读到最新值,但读这个动作本身很贵。该优化消除了其中大部分跨核读。

3.2 机制

生产者持有一个私有的、非原子的 `cached_head_`。大多数 `push` 用这个缓存值判断是否已满,只触碰生产者自己的 L1。只有当缓存值显示“已满”时,生产者才执行一次真正的 `head_.load()` 去刷新。

```
bool try_push(const T& item) {
    size_t t = tail_.load(std::memory_order_relaxed);
    if (t - cached_head_ == Capacity) {           // cheap local test
        cached_head_ = head_.load(std::memory_order_acquire); // refresh
        if (t - cached_head_ == Capacity)
            return false;                         // genuinely full
    }
    buf_[t & (Capacity - 1)] = item;
    tail_.store(t + 1, std::memory_order_release);
    return true;
}
```

3.3 安全性

引理(缓存单调性)。在任何时刻, $\text{cached_head_} \leq \text{actual_head}$, 其中 `actual_head` 是 `head_` 的当前值。

证明。 `cached_head_` 只可能被某次对 `head_` 的过往加载所赋值。由于消费者单调地递增 `head_`, 任何先前观察到的值都 $\leq$ 当前值。 $\square$

命题(保守正确性)。若生产者写入某个槽位,则该槽位确实空闲;生产者绝不覆盖未读数据。

证明。生产者仅当其满判断失败时才写入,即(可能刷新之后) $t - \text{cached_head_} < \text{Capacity}$ 。由单调性引理, $t - \text{actual_head} \leq t - \text{cached_head_} < \text{Capacity}$ 。因此真实占用严格低于容量,故存在空闲槽位。缓存值只可能造成虚假已满(一次保守的拒绝),绝不会造成虚假空闲;“假满”无害(它触发一次刷新),而会导致数据损坏的“假空”不可能发生。 $\square$

3.4 它不做什么

该优化并不节流或控制生产者的填充速率,也不支配缓冲区填满的快慢。填充速率完全由生产者与消费者的相对吞吐决定。缓存索引改变的只是满判断这一检查的成本——用偶尔一次的跨核加载替代频繁的跨核加载。生产者最终受限于物理容量边界,而非缓存。打个比方:缓存索引是一把更便宜的量尺,用来读取水位;换量尺并不改变水流入或流出的快慢。

4. 无符号硬件溢出下的正确性

本节形式化地说明:当 k 位计数器溢出回绕时,绝对计数设计为何仍然正确。我们先确立算法所维持的不变量,再在该不变量下证明寻址不变性与距离不变性。

4.1 形式化设定

设 $\mathbb{Z}_{2^k}$ 为模 2^k 的整数环(一个 k 位无符号寄存器,通常 $k = 64$)。无符号加减定义为 $a \oplus b \equiv a + b \pmod{2^k}$ 与 $a \ominus b \equiv a - b \pmod{2^k}$ 。设 $N = 2^m$, 其中 $0 < m < k$ 为容量,寻址映射 $f: \mathbb{Z}_{2^k} \rightarrow \mathbb{Z}_N$ 为位截断 $f(x) = x \text{ AND } (2^m - 1)$ 。

引理(f 的模同态性)。由于掩码 $(2^m - 1)$ 隔离出低 m 位,故 $f(x) \equiv x \pmod{2^m}$ 。又因 $2^m \mid 2^k$, 该映射可降到商环并保持加法结构: $f(a \oplus b) \equiv (a + b \pmod{2^k}) \pmod{2^m} \equiv (a + b) \pmod{2^m} \equiv (f(a) + f(b)) \pmod{2^m}$ 。

4.2 所维持的不变量

设 $t, h \in \mathbb{Z}_{2^k}$ 为生产者与消费者的绝对计数。

不变量(无溢出占用)。在每一个可达状态下, $0 \leq (t \ominus h) < N$ 。

后续所有正确性都依赖这个不变量,因此我们证明算法维持它,而不是假设它。

定理 1(不变量维持)。在单生产者单消费者运行下,不变量 $0 \leq t \ominus h < N$ 在每个可达状态下均成立。

证明。对操作做归纳。

基例。初始时 $t = h = 0$, 故 $t \ominus h = 0$, 且 $0 \leq 0 < N$ 。

归纳步。假设 $0 \leq t \ominus h < N$ 。两种操作可改变状态。

Push。生产者仅在其守卫验证 $t \ominus h < N$ (即 $t \ominus h \leq N - 1$) 之后才推进 $t \mapsto t \oplus 1$ 。递增后占用量变为 $(t \oplus 1) \ominus h = (t \ominus h) + 1 \leq (N - 1) + 1 = N$ 。结合下界,新占用量落在 $[1, N]$ 内,处于可接受范围。(在极值 N 处缓冲区恰好满,下一次 push 守卫会正确拒绝。)只有生产者修改 t , 故 h 不变,下界 $0 \leq t \ominus h$ 得以保持。

Pop。消费者仅在验证 $t \ominus h > 0$ (非空)之后才推进 $h \mapsto h \oplus 1$ 。递增后 $t \ominus (h \oplus 1) = (t \ominus h) - 1 \geq 0$, 下界得以保持;占用量严格减小,故上界亦保持。只有消费者修改 h , 故 t 不变。

由于生产者只改 t 、消费者只改

h , 两个更新从不在同一计数器上竞争,且各自都保持两条边界。因此该不变量在所有可达状态下成立。□

4.3 寻址不变性

定理 2(溢出处寻址无缝)。当生产者计数 $t_1 = 2^k - 1$ 递增到 $t_2 = t_1 \oplus 1 = 0$ 时,物理索引仍保持连续: $f(t_2) = (f(t_1) + 1) \pmod{N}$ 。

证明。左端。 $t_2 = 0$, 故 $f(t_2) = f(0) \equiv 0 \pmod{2^m}$, 又因 $0 \in [0, 2^m - 1]$, 得 $f(t_2) = 0$ 。

右端。 $(f(t_1) + 1) \pmod{2^m} \equiv (t_1 + 1) \pmod{2^m} \equiv (2^k - 1 + 1) \pmod{2^m} \equiv 2^k \pmod{2^m}$ 。因 $2^m \mid 2^k$, 此值为 0。

两端皆等于 0,由相等的传递性得 $f(t_2) = (f(t_1) + 1) \bmod N = 0$ 。绝对计数的回绕在物理槽位序列上不产生任何不连续。 $\square$

4.4 距离不变性

定理 3(经环算术的距离不变性)。 设生产者已回绕,使其名义值低于消费者: $t_2 = 0 < h$,其中 $h = 2^k - d$, d 为真实物理距离, $0 < d < N$ 。则无符号减法无需分支即得精确距离: $t_2 \ominus h = d$ 。

证明。在 $\mathbb{Z}_{2^k}$ 中,减法即加上加法逆元: $t_2 \ominus h \equiv (t_2 + 2^k - h) \bmod 2^k = (0 + 2^k - (2^k - d)) \bmod 2^k = d \bmod 2^k$ 。由所维持的不变量, $0 < d < N < 2^k$,故 d 落在基本域内,外层约化为恒等: $d \bmod 2^k = d$ 。于是 $t_2 \ominus h = d$ 。 $\square$

注记(补码是实现,而非定义)。 因 t 、 h 为无符号,实际起作用的代数是模 2^k 的环算术。项 $2^k - h$ 是 h 在此环中的加法逆元;其位模式恰好与补码编码($\text{NOT } h + 1$)一致,这正是为何 ALU 上一次无符号减法即便在数值上 $t < h$ 时也能给出正确距离。硬件免费地纠正了这种表面上的“时间倒置”。

4.5 位流演算示例

设 $k = 8(2^k = 256)$ 与 $N = 8$ 。设 $h = 250(1111\ 1010)$,生产者回绕到 $t = 0(0000\ 0000)$;真实距离为 $256 - 250 = 6$ 。

- h 的加法逆元: $\text{NOT } 1111\ 1010 + 1 = 0000\ 0101 + 1 = 0000\ 0110$ (值 6)。
- 无符号加 $t + (2^k - h)$: $0000\ 0000 + 0000\ 0110 = 0000\ 0110$ (值 6)。
- 掩码 $\& (N-1) = \& 0000\ 0111$: $0000\ 0110 \text{ AND } 0000\ 0111 = 0000\ 0110$ (槽位 6)。

模算术的循环拓扑以零分支、零误预测惩罚吸收了这次回绕。

5. 综述

SPSC 环形缓冲区奖励在恰当抽象层级上的推理。顺序的正确性归结为一个问题——加载线程之后是否会去读另一线程发布的载荷?——这个问题在使用处无需援引 store buffer 或 MESI,就决定了 acquire 还是 relax。缓存索引的正确性归结为单调性:一个陈旧的对面计数器只会朝保守方向出错。寻址的正确性归结为两条模运算事实:掩码是一个同态,无符号减法就是环减法。底层的硬件机制——store buffer、缓存一致性、投机执行——解释了这些抽象为何成立,但实际工作的工程师所推理的,是“数据 + 旗子”的 happens-before、单调计数器与同余。代码正是在这一层级上被写就,也在这一层级上被辩护。